

Weather Anomaly Detector

MSITM-6341-AIP01- Python

Course Faculty - Dennis Wang

Technical Documentation

GitHub Repository:

<https://github.com/sondebisola/GroupA--MAITM6341SP25Finals>

Role	Team Member
Data Pipeline Lead	Bisola
ML Lead	Dibyanshi
Flask Backend	Sruthi
Frontend and Dashboard	Ravikumar

1. Project Overview

The Weather Anomaly Detector is a full-stack machine learning application that identifies statistically unusual weather patterns for any city worldwide. A user enters a city name through the web interface and receives an interactive dashboard showing anomalous days flagged by an Isolation Forest model. The model is trained on five years of historical weather data spanning 2020 through 2024.

Core Capabilities

- Accepts any city name worldwide and resolves its coordinates automatically via the Open-Meteo geocoding API, which requires no API key.
- Fetches five years of daily historical weather data, covering 1,827 days per city, including maximum temperature, minimum temperature, precipitation, and wind speed.
- Engineers 13 input features per day, including raw metrics, 30-day rolling deviations, and z-scores, before passing data to the model.
- Detects anomalous days using an unsupervised Isolation Forest algorithm configured with a 5 percent contamination rate.
- Renders six interactive Plotly charts in a tabbed, lazy-loaded browser dashboard for exploring results.
- Exposes both a browser-based UI and a REST JSON API endpoint at /api/analyze for programmatic access.

2. Team Responsibilities

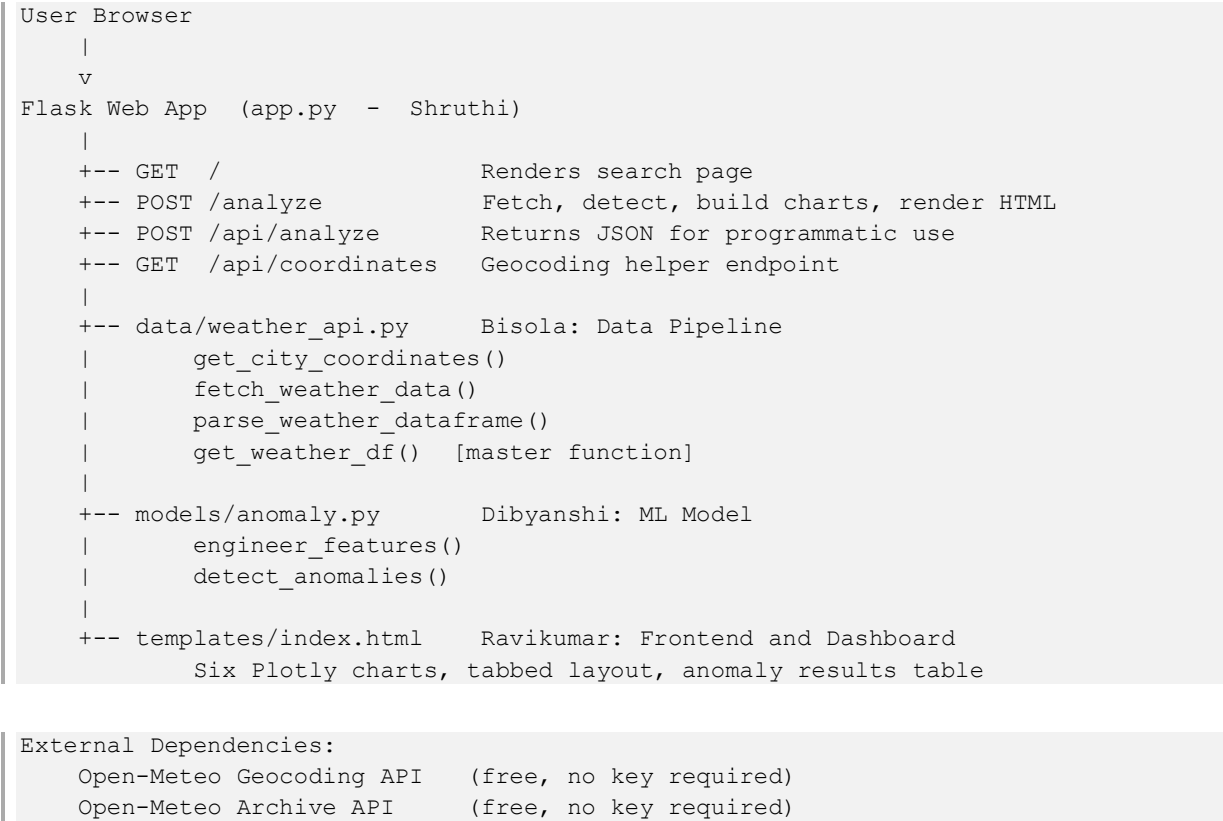
The project was divided into four clearly scoped workstreams. Each team member owned one layer of the application, with defined interfaces between layers to enable parallel development.

Team Member	Role	Module	Scope
Bisola	Data Pipeline Lead	weather_api.py	Geocoding, API fetch, DataFrame cleaning
Dibyanshi	ML Lead	anomaly.py	Feature engineering, Isolation Forest model, anomaly scoring
Sruthi	Flask Backend	app.py	HTTP routes, Flask-to-module integration, JSON API, result rendering
Ravikumar	Frontend and Dashboard	index.html, style.css	Plotly chart construction, tabbed UI, parallel coordinates, result display

3. System Architecture

The application follows a clean three-layer design. Flask serves as the integration layer, importing from the data pipeline module written by Bisola and the ML model module written by Dibyanshi. Shruthi built the route logic and result assembly. Ravikumar built the frontend charts and dashboard layout.

Architecture Overview



4. Technology Stack

Layer	Technology	Purpose
Language	Python 3.x	All backend logic
Web Framework	Flask	HTTP routing and Jinja2 templating
Data	pandas, numpy	DataFrame manipulation and numerical operations
Machine Learning	scikit-learn	Isolation Forest model and StandardScaler
Visualization	Plotly	Interactive browser charts

Layer	Technology	Purpose
Weather Data	Open-Meteo API	Free historical weather archive and geocoding
Development	Google Colab	Collaborative notebook execution
Version Control	GitHub	Code sharing across team members
Public Tunnel	ngrok	Exposing local Flask server to a public URL

Python Dependencies

requirements.txt contents:

```
flask
pandas
numpy
scikit-learn
plotly
requests
```

5. Module Breakdown

5.1 Data Pipeline Module (Bisola)

File: weather-anomaly-detector/data/weather_api.py

This module is the single source of weather data for the entire application. The Flask route only needs to call one master function, `get_weather_df()`. All geocoding, fetching, and cleaning steps are encapsulated internally.

`get_city_coordinates(city_name)`

Returns: (latitude, longitude, display_name)

Calls the Open-Meteo geocoding endpoint to convert a freeform city name string into geographic coordinates. If the city cannot be found, the function raises a `ValueError`, which the Flask route catches and displays as a user-facing error message.

```
url = "https://geocoding-api.open-meteo.com/v1/search"
params = {"name": city_name, "count": 1, "language": "en", "format": "json"}
```

`fetch_weather_data(lat, lon, start_year=2020, end_year=2024)`

Returns: dict (raw JSON response from the Open-Meteo archive API)

Retrieves four daily weather variables for the given coordinates and date range:

- `temperature_2m_max`: daily maximum temperature in degrees Celsius
- `temperature_2m_min`: daily minimum temperature in degrees Celsius
- `precipitation_sum`: total daily precipitation in millimetres
- `windspeed_10m_max`: maximum daily wind speed in kilometres per hour

The timezone parameter is set to auto so that dates align with the local timezone of the requested city.

parse_weather_dataframe(raw, city_name)

Returns: pd.DataFrame

Converts raw API JSON into a clean, date-indexed DataFrame. Three deliberate design decisions were made during this step:

- Date indexing: set_index('date') is applied so that the date becomes the row label. This is standard practice for time-series DataFrames and produces cleaner Plotly chart axes.
- Derived column: a temp_range column is added by subtracting daily minimum temperature from daily maximum temperature. Unusually wide temperature swings are a meaningful independent anomaly signal that the model can exploit.
- Missing value strategy: fillna(median()) is used rather than fillna(mean()), because extreme values present in the data would skew the mean and corrupt any imputed values.

get_weather_df(city_name, start_year=2020, end_year=2024)

Returns: (df, display_name)

This is the master function that chains geocoding, fetching, and parsing into a single call. It is the only function imported and called by app.py. All other functions in this module are internal pipeline steps.

5.2 Machine Learning Module (Dibyanshi)

File: weather-anomaly-detector/models/anomaly.py

This module handles all machine learning work: feature engineering and unsupervised anomaly detection. It exposes two clean functions for the Flask backend to call.

engineer_features(df)

Returns: enriched pd.DataFrame with 8 additional columns, giving 13 total model input features.

Feature Group	Columns Added	Purpose
30-day rolling averages	temp_max_30d, precip_30d, wind_30d	Captures the seasonal baseline for each variable
Deviations from rolling average	temp_max_dev, precip_dev, wind_dev	Measures how far each day departs from its seasonal norm
Z-scores	temp_max_z, temp_min_z, precip_z, wind_z, temp_range_z	Standardises all features to a common numeric scale

NaN values introduced by the rolling window edges are filled with column medians before the feature matrix is passed to the model.

detect_anomalies(df, contamination=0.05)

Returns: DataFrame with three new columns appended:

- `is_anomaly` (boolean): True when the day is flagged as anomalous by the model
- `anomaly_score` (float): raw model output score; more negative values indicate a greater degree of anomaly
- `anomaly_label` (integer): -1 for anomaly, +1 for normal, following the scikit-learn convention

Model configuration used in this function:

```
IsolationForest(  
    contamination = 0.05,    # expect approximately 5 percent anomalous days  
    n_estimators  = 200,    # 200 decision trees in the ensemble  
    random_state  = 42,    # ensures reproducible results  
    n_jobs        = -1      # uses all available CPU cores  
)
```

Why Isolation Forest?

Isolation Forest is an unsupervised anomaly detection algorithm that works by randomly selecting a feature and a random split value within its range. Points that occupy sparse regions of feature space are isolated by fewer splits and therefore receive lower anomaly scores. The algorithm requires no labeled training data, which is appropriate here because no ground-truth anomaly labels exist for historical weather data.

Why contamination = 0.05?

Setting contamination too low, for example at 0.01, causes the model to be too conservative and miss genuine weather anomalies. Setting it too high, for example at 0.20, causes the model to flag too many days, reducing the practical meaning of each flag. A value of 0.05 implies approximately 18 anomalous days per year, which is consistent with real-world extreme weather event frequency and serves as a well-established baseline for daily time-series anomaly detection.

Why StandardScaler Before Training?

Isolation Forest selects features randomly during tree construction. Without prior scaling, features with numerically large ranges, such as wind speed in km/h, would dominate the random split selection over features with smaller ranges, such as precipitation in mm. StandardScaler normalises all features to a mean of zero and a standard deviation of one before training, eliminating this imbalance.

Anomaly Score Interpretation

- Score greater than 0: the day is within the normal data distribution
- Score less than 0: the day is anomalous, existing in a sparse region of feature space
- Score much less than 0: the day is highly anomalous, representing a rare multi-variable combination

5.3 Flask Backend (Sruthi)

File: weather-anomaly-detector/app.py

The Flask backend integrates the data pipeline module and the ML module into a web-accessible service. Shruthi built the route logic, error handling, result assembly, and JSON API.

Method	Route	Description
GET	/	Renders the landing page with the city input form
POST	/analyze	Core route: geocodes city, fetches data, runs anomaly detection, builds charts, renders HTML response
POST	/api/analyze	JSON REST API version of /analyze, returning structured data for programmatic use
GET	/api/coordinates	Geocoding helper that accepts a city query parameter and returns latitude, longitude, and display name as JSON
GET	/debug	Development-only sanity check endpoint; returns a sample of raw weather data as JSON

Processing Logic for the /analyze Route

The route follows this sequence on every POST request:

1. Read and validate form inputs: city name and view_year (either 'all' or a specific year from 2020 to 2024).
2. Always train the model on the full 2020-2024 dataset of 1,827 days. A single year is too small for robust Isolation Forest training.
3. Filter display data to the selected year after model training, if a specific year was chosen. The model itself still used all five years.
4. Compute summary statistics: average and extreme temperatures, total precipitation, average wind speed, and anomaly count.
5. Build six Plotly figures and serialise each to JSON using plotly.utils.PlotlyJSONEncoder for passing to the Jinja2 template.
6. Render index.html with all context variables: summary stats, anomaly records, weather records, and chart JSON.

JSON API Usage Example

```
curl -X POST http://localhost:5000/api/analyze \
-H "Content-Type: application/json" \
-d '{"city": "Lagos", "view_year": "2023"}'
```

Example response structure:

```
{
  "summary": {
    "city": "Lagos, Nigeria",
    "year": "2023",
    "total_days": 365,
    "avg_temp_max": 31.4,
```

```
    "anomaly_count": 18
  },
  "anomalies": [
    {"date": "2023-04-17", "temp_max": 38.2, ...}
  ]
}
```

5.4 Frontend and Dashboard (Ravikumar)

Files: weather-anomaly-detector/templates/index.html, static/style.css

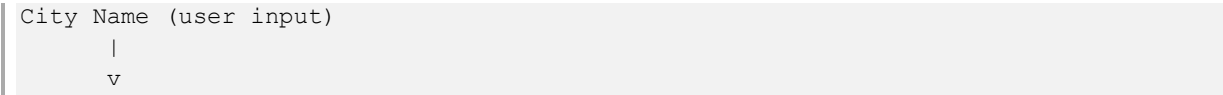
Ravikumar built the complete browser-facing experience, including all six Plotly chart types, the tabbed navigation system, the summary statistics display, and the anomaly results table. Charts are lazy-rendered: JavaScript only calls Plotly.newPlot() when a user clicks the corresponding tab. This avoids six simultaneous renders on page load and keeps initial load time fast.

Chart	What It Shows
Anomaly Score Over Time	Continuous Isolation Forest output plotted as a line. Days flagged as anomalous are highlighted with red markers below the score = 0 threshold.
Temperature, Precipitation, and Score Stack	A three-panel stacked chart showing temperature trend lines, daily precipitation bars, and anomaly scores, all aligned on a shared date axis.
Precipitation Distribution	Histogram of daily rainfall totals. Mean and median reference lines are overlaid to show the skew toward heavy-rain days.
Windspeed Trends	Daily wind speed plotted in grey with a 30-day centred rolling average overlaid in orange to reveal seasonal patterns.
Feature Correlation Heatmap	Pearson correlation between the five base weather features. Uses a diverging red-blue colour scale to show positive and negative correlations.
Parallel Coordinates	Each line represents one day drawn across four variable axes. Anomalous days are rendered in red. Axis ranges can be dragged to filter and isolate specific subsets.

The landing page also displays an informational strip showing key model statistics (1,827 training days, 5 percent contamination, 200 estimators, 13 features, and 6 chart types) and a row of example city buttons that auto-submit the form on click.

6. Data Pipeline

Data flows from a user-entered city name through geocoding, API retrieval, and cleaning before being passed to the machine learning module.



```
Open-Meteo Geocoding API
  Returns: latitude, longitude, display name
  |
  v
Open-Meteo Archive API
  Returns: 1,827 daily records (2020-2024)
  Variables: temp_max, temp_min, precipitation, windspeed
  |
  v
parse_weather_dataframe()
  Sets date index, adds temp_range column, fills missing values with median
  |
  v
Clean DataFrame (1,827 rows x 6 columns) ready for feature engineering
```

Open-Meteo provides ERA5-Land reanalysis data, which is gap-free for the variables used. Missing values are rare but are filled with column medians as a precaution. Setting timezone to auto ensures that all dates align with the local timezone of the city, avoiding off-by-one date issues near midnight.

7. Feature Engineering

The raw 5-column dataset is insufficient for multi-variable anomaly detection on its own. The 13-feature matrix built by Dibyanshi provides the model with rich context about each day's behaviour relative to its seasonal trend, not just its absolute value.

Feature Rationale

Feature	Rationale
temp_max, temp_min, precipitation, windspeed, temp_range	Raw daily observations from Open-Meteo. These are the baseline signal.
temp_max_30d, precip_30d, wind_30d	30-day rolling averages that represent the seasonal context for each variable.
temp_max_dev, precip_dev, wind_dev	Daily deviation from the rolling average. A day that departs significantly from its seasonal norm is a stronger anomaly candidate than a day that merely has a high absolute value.
temp_max_z, temp_min_z, precip_z, wind_z, temp_range_z	Z-scores computed across the full history of each variable. These put all features on a common numeric scale before the StandardScaler step.

Why a 30-Day Rolling Window?

A 7-day window is too narrow: it would flag normal week-to-week weather variation as anomalous. A 90-day window is too wide: it blurs seasonal transitions and loses sensitivity to month-level patterns. A 30-day centred window balances responsiveness with stability for daily weather data.

8. Web Application and API Routes

8.1 Request Flow for /analyze

```
User submits city name and year selection
  | POST /analyze
  v
Flask reads form inputs
  |
  v
get_weather_df(city) [Bisola's module]
  Returns clean DataFrame for 2020-2024
  |
  v
detect_anomalies(df) [Dibyanshi's module]
  Returns DataFrame with is_anomaly, anomaly_score, anomaly_label
  |
  v
Filter to selected year if applicable
  |
  v
Compute summary statistics
  |
  v
Build six Plotly figures, serialise to JSON
  |
  v
render_template('index.html', ...) [Ravikumar's frontend]
  |
  v
Browser: Plotly.newPlot() on each tab click (lazy-loaded)
```

8.2 JSON API Endpoint

The `/api/analyze` endpoint accepts a JSON body and returns structured data without any HTML rendering. This is useful for integrations or testing without a browser.

Request format:

```
{ "city": "Houston", "view_year": "2022" }
```

Response format:

```
{ "summary": { "city": "...", "total_days": 365, "anomaly_count": 18, ... },
  "anomalies": [ { "date": "2022-08-21", "temp_max": 41.3, ... } ] }
```

9. Project Setup and Deployment

9.1 Running in Google Colab

```
# Step 1: Clone the repository
!git clone https://github.com/sondebisola/GroupA--MAITM6341SP25Midterm.git
%cd GroupA--MAITM6341SP25Midterm
```

```
# Step 2: Install Python dependencies
!pip install flask pandas numpy scikit-learn plotly requests pyngrok
```

```
# Step 3: Start Flask in a background thread
from threading import Thread
Thread(target=lambda: app.run(port=5000, use_reloader=False)).start()
```

```
# Step 4: Create a public tunnel with ngrok
from pyngrok import ngrok
ngrok.set_auth_token("YOUR_TOKEN_HERE")
public_url = ngrok.connect(5000)
print(public_url)
```

9.2 Project Directory Structure

weather-anomaly-detector/	
app.py	Flask application (Shruthi)
requirements.txt	Python dependencies
data/	
weather_api.py	Data pipeline module (Bisola)
models/	
anomaly.py	Machine learning module (Dibyanshi)
templates/	
index.html	Jinja2 HTML template (Ravikumar)
static/	
style.css	Stylesheet (Ravikumar)

10. Limitations and Future Work

10.1 Current Limitations

- No result caching: every request re-fetches weather data and retrains the model from scratch. Repeated queries for the same city are slow and generate unnecessary API calls.
- Unsupervised model only: there are no labeled ground-truth anomaly records to validate model accuracy against. The anomaly rate is controlled by the contamination hyperparameter rather than learned from labeled examples.
- API rate limits: the Open-Meteo free tier may throttle requests under heavy traffic, causing fetch failures.
- Non-persistent deployment: the Colab and ngrok setup resets each time the session ends. The application is not suitable for always-on production use in this form.

10.2 Potential Future Improvements

- Implement Redis or SQLite caching so that repeated queries for the same city return pre-computed results without hitting the API again.
 - Integrate labeled historical extreme weather event datasets such as the NOAA Storm Events database to enable supervised model validation and evaluation metrics.
 - Deploy to a persistent cloud platform such as Render, Railway, or AWS to provide always-on public access without a tunnel.
 - Add a multi-city comparison mode that allows users to run and display anomaly results for two or more cities side by side.
 - Extend the data pipeline to hourly weather data to enable intra-day anomaly detection.
 - Add an email or webhook notification system that alerts users when newly anomalous days are detected for a saved city.
-